

CAPSTONE PROJECT

FinAssist AI

An Intelligent Personal Finance Management Platform using Agentic AI

Enterprise AI Capstone Project

Client Organization: FinWise Technologies Pvt. Ltd.

Project Duration: 2 Days

Team Size: 4 Members

Difficulty Level: Advanced

Technology Stack

Category	Technologies
Programming Language	Python
Agent Framework	LangChain
Workflow Engine	LangGraph
Large Language Model	OpenAI / Gemini / Ollama (Llama 3.2)
User Interface	Streamlit
Data Processing	Pandas
Visualization	Matplotlib
Validation	Pydantic
AI Concepts	Multi-Agent System, Memory, Reflection, Tool Calling

Project Theme

Building an Intelligent Multi-Agent Financial Assistant

FinWise Technologies aims to transform traditional personal finance management by introducing an Agentic AI-powered financial assistant capable of understanding customer spending behavior, planning personalized budgets, forecasting future expenses, and providing proactive financial guidance.

Unlike conventional chatbots, the proposed system will consist of multiple collaborating AI agents that communicate through a centralized orchestration workflow. Each agent specializes in a specific financial domain while sharing contextual memory and reflecting on intermediate results before generating a final recommendation.

Students will design and implement this solution using LangGraph-based multi-agent orchestration, enabling autonomous reasoning, tool usage, shared memory, and explainable financial decision-making.

Table of Contents

Section	Page
1. Business Background	2
2. Client Profile	2
3. Existing Business Challenges	2
4. Project Objective	3
5. Functional Requirements	3
6. AI System Overview	3
7. Multi-Agent Architecture	4
8. AI Agent Responsibilities	4
9. Technical Requirements	5
10. Suggested Project Structure	5
11. Implementation Roadmap	5
12. Deliverables	6
13. Evaluation Criteria	6
14. Bonus Challenges	6
15. Future Enhancements	6

Learning Outcomes

- Design enterprise-scale Agentic AI applications.
- Build and coordinate multiple AI agents using LangGraph.
- Implement shared memory for context-aware conversations.
- Apply reflection techniques to improve AI responses.
- Develop modular production-ready Python applications.
- Create interactive AI interfaces using Streamlit.

Client Expectation

The client expects a Minimum Viable Product (MVP) capable of demonstrating intelligent financial reasoning, multi-agent collaboration, personalized recommendations, expense forecasting, budget planning, explainable AI decisions, and enterprise-ready software architecture.

Business Background & Problem Statement

1. Business Background

About the Client

FinWise Technologies Pvt. Ltd. is one of India's rapidly growing financial technology companies specializing in intelligent personal finance management solutions. The organization provides digital services that help customers manage expenses, monitor savings, create financial goals, and receive personalized financial recommendations.

The company serves more than 250,000 active customers and processes over 1.2 million financial transactions every day. To improve customer experience and scale advisory services, FinWise plans to introduce an Agentic

AI platform capable of understanding spending behaviour, forecasting expenses, and providing proactive financial guidance.

Current Business Environment

- Transaction History
- Expense Reports
- Budget Tracking
- Savings Dashboard
- Monthly Statements
- Financial Goal Management
- Notification System
- Customer Support Portal

Although these systems provide valuable information, they operate independently. Users must manually interpret reports before making financial decisions, resulting in poor financial awareness and inconsistent planning.

Existing Technology Landscape

Existing System	Purpose
Transaction Management System	Stores customer transactions
Expense Tracking Service	Records categorized expenses
Budget Management Portal	Monthly budget creation
Savings Dashboard	Displays savings progress
Financial Goal Manager	Tracks long-term goals
Notification Service	Payment reminders
Customer Support Portal	Handles customer queries

Current Business Challenges

Customer Challenges

- Difficulty identifying unnecessary spending.
- Lack of personalized budgeting recommendations.
- Limited visibility into recurring expenses.
- No prediction of future financial commitments.
- Generic financial advice that ignores personal circumstances.

Business Challenges

- Manual financial analysis by advisors.
- Delayed financial recommendations.
- High operational costs.
- Limited scalability.
- Inconsistent advisory services.

Executive Goals

The executive leadership expects FinAssist AI to:

- Deliver personalized financial guidance.
- Reduce manual effort through AI automation.
- Improve customer engagement with proactive recommendations.
- Enable predictive financial planning.
- Provide explainable AI-driven decisions.

Why Agentic AI?

Traditional chatbots answer questions but cannot independently plan tasks, collaborate across specialized components, or learn from previous interactions. FinAssist AI will adopt a multi-agent architecture coordinated by LangGraph, where specialized agents share memory, use tools, and perform reflection before delivering the final response. This architecture enables intelligent task routing, context-aware conversations, personalized recommendations, and transparent decision-making.

Project Scope, Objectives & Functional Requirements

3. Project Objective

Project Vision

FinWise Technologies intends to build FinAssist AI, an intelligent personal finance management platform that leverages Agentic AI to help users make informed financial decisions. Unlike traditional finance applications that only display historical data, FinAssist AI will proactively analyze spending, recommend personalized budgets, forecast future expenses, and provide explainable financial guidance through collaboration among multiple AI agents.

The solution should demonstrate enterprise-grade architecture using LangChain, LangGraph, Memory, Reflection, and Tool Calling while remaining modular and scalable.

4. Project Scope

The objective of this capstone is to design and implement a Multi-Agent AI System that automates personal financial planning. The platform should support expense analysis, budget planning, financial advisory, expense forecasting, reporting, and goal-based recommendations through a conversational Streamlit interface.

In Scope

- Upload and process financial transaction history.
- Automatically categorize expenses.
- Generate monthly spending summaries.
- Create personalized budgets.
- Recommend savings opportunities.
- Forecast recurring and future expenses.
- Support natural language financial conversations.

Out of Scope

- Real banking integration
- Online payment processing
- Cryptocurrency trading
- Tax filing
- Loan approval systems
- Insurance claim processing

5. Functional Requirements

Requirement ID	Description
FR-01	Upload financial transaction data (CSV/Excel).
FR-02	Automatically categorize expenses.
FR-03	Calculate income, expenses and savings.
FR-04	Generate personalized monthly budgets.
FR-05	Forecast future expenses.
FR-06	Recommend savings opportunities.
FR-07	Maintain conversation history using memory.
FR-08	Route requests to the correct AI agent.
FR-09	Validate responses using reflection.
FR-10	Display financial insights in Streamlit.

6. Business Rules

- Expenses shall be categorized into Food, Transportation, Utilities, Shopping, Entertainment, Healthcare, Education and Miscellaneous.
- Budget recommendations must consider income, current expenses, financial goals and savings.
- Expense forecasting should use historical transaction data.
- Recommendations should include a short explanation for transparency.
- A reflection step must verify completeness and consistency before the final response.

7. User Stories

User Story 1: As a customer, I want to upload my transaction history so that I can understand my spending habits.

User Story 2: As a customer, I want AI to generate a personalized monthly budget.

User Story 3: As a customer, I want to know if I am likely to exceed my budget.

User Story 4: As a customer, I want recommendations to reduce unnecessary expenses.

User Story 5: As a customer, I want the AI assistant to remember my financial goals.

8. Success Criteria

- Natural language interaction with the AI assistant.
- Correct routing through the Supervisor Agent.
- Effective collaboration among all four AI agents.
- Conversation memory maintained across interactions.

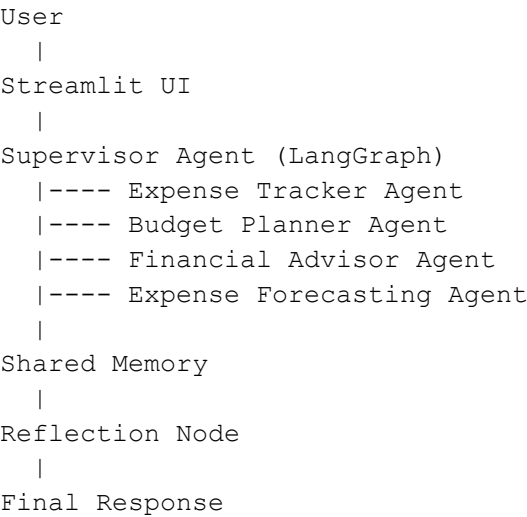
- Reflection improves recommendation quality.
- Production-ready modular architecture with Streamlit UI.

Multi-Agent Architecture & System Design

9. AI Solution Architecture

FinAssist AI follows a Supervisor-based Multi-Agent Architecture implemented using LangGraph. A Supervisor Agent receives every user request, determines the intent, routes the request to one or more specialized agents, aggregates the results, validates them through a Reflection Node, and returns a final response.

High-Level Workflow



10. AI Agent Responsibilities

Agent	Primary Responsibility	Inputs	Outputs	Tools
Supervisor	Intent detection & routing	User query	Execution plan	LangGraph
Expense Tracker	Categorize expenses	Transactions	Expense insights	Pandas
Budget Planner	Generate budgets	Income, expenses	Budget plan	Calculator
Financial Advisor	Savings recommendations	Financial profile	Advice	LLM
Forecasting	Predict future expenses	Historical data	Forecast	Pandas/LLM

11. Memory Strategy

- Maintain conversation history across user sessions.
- Remember financial goals and preferences.
- Reuse previous analysis to avoid repeated processing.
- Share context across all four agents through a common state.

12. Reflection Strategy

Before returning a response, the Reflection Node reviews the combined output of all participating agents to ensure correctness, completeness, and consistency.

- Verify that recommendations are supported by available financial data.
- Check for missing information or conflicting advice.
- Improve clarity and explainability of the final response.
- Return the validated response to the user.

13. Tool Calling Strategy

Tool	Purpose
CSV Reader	Load transaction history
Calculator	Financial calculations
Visualization Tool	Generate spending charts
Memory Store	Persist conversation context
LLM	Reasoning and response generation

14. Example Execution Flow

1. User uploads six months of transactions and asks for savings advice.
2. Supervisor identifies multiple tasks and routes them to the Expense Tracker, Budget Planner, and Financial Advisor.
3. Agents analyze data independently and share results through LangGraph state.
4. Reflection Node validates the recommendations.
5. Final consolidated response is displayed in the Streamlit interface.

Implementation Roadmap, Technical Guidelines & Project Organization

15. Technical Requirements

Category	Required Technology
Programming Language	Python 3.11+
LLM Framework	LangChain
Agent Orchestration	LangGraph
Large Language Model	OpenAI / Gemini / Ollama (Llama 3.2)
Frontend	Streamlit
Data Processing	Pandas
Visualization	Matplotlib
Data Validation	Pydantic
Version Control	Git & GitHub
Optional Database	SQLite / ChromaDB

16. Recommended Project Folder Structure

```
finassist-ai/
├── app.py
├── supervisor.py
├── graph.py
├── requirements.txt
├── README.md
├── agents/
│   ├── expense_tracker.py
│   ├── budget_planner.py
│   ├── financial_advisor.py
│   └── forecasting_agent.py
├── memory/
│   ├── conversation_memory.py
│   └── state.py
├── prompts/
├── tools/
│   ├── csv_reader.py
│   ├── calculator.py
│   └── visualization.py
├── data/
├── tests/
└── utils/
```

17. Implementation Roadmap

Phase 1 – System Design

Identify business requirements, define the supervisor workflow, design the multi-agent architecture, and prepare the project structure.

Phase 2 – AI Agent Development

Develop each of the four AI agents independently with clearly defined responsibilities, prompts, and tool integrations.

Phase 3 – LangGraph Integration

Integrate all agents using LangGraph. Implement routing logic, shared state, and agent collaboration.

Phase 4 – Memory & Reflection

Implement conversation memory, maintain user context, and introduce a reflection node to validate recommendations before responding.

Phase 5 – Streamlit Application

Develop an intuitive user interface for uploading transaction data, interacting with the AI assistant, and visualizing financial insights.

Phase 6 – Testing & Deployment

Perform functional testing, validate agent collaboration, document the project, and prepare the final demonstration.

18. Team Responsibilities

Team Member	Primary Responsibility
Member 1	Streamlit UI, Prompt Engineering, User Experience
Member 2	Financial Tools, CSV Processing, Visualization
Member 3	LangGraph Workflow, Memory Integration, Agent Routing
Member 4	Reflection, Testing, Documentation, Project Integration

19. Development Best Practices

- Use modular Python code with reusable components.
- Follow object-oriented programming principles where appropriate.
- Write meaningful docstrings and inline comments.
- Use environment variables to store API keys securely.
- Validate user input before processing.
- Implement proper exception handling and logging.
- Maintain clean Git commit history throughout development.
- Ensure each AI agent has a single, well-defined responsibility.

20. Expected Learning Outcomes

- Design enterprise-scale Agentic AI applications.
- Build production-ready LangGraph workflows.
- Develop collaborative AI agents with memory and reflection.
- Create modular AI systems using LangChain.
- Deploy interactive AI solutions using Streamlit.

Project Deliverables, Evaluation & Client Acceptance

21. Expected Deliverables

- Working Streamlit application demonstrating the complete multi-agent workflow.
- Complete Python source code with modular project structure.
- LangGraph workflow implementation with Supervisor Agent.
- Architecture diagram illustrating agent interactions and memory flow.
- Technical documentation and README with setup instructions.
- Presentation (10–15 minutes) demonstrating business value and technical implementation.
- GitHub repository with clean commit history.
- Sample transaction dataset and test scenarios.

22. Evaluation Criteria (100 Marks)

Assessment Criteria	Marks
Problem Understanding & Solution Design	10
Prompt Engineering	10
LangChain & Tool Integration	10
LangGraph Workflow	15
Multi-Agent Collaboration	15
Memory & Reflection	10
Streamlit User Interface	10
Code Quality & Documentation	10
Testing & Demonstration	5
Innovation & Bonus Features	5
Total	100

23. Bonus Enterprise Challenges (Optional)

- Goal-based financial planning dashboard.
- OCR-based receipt processing.
- Voice-enabled financial assistant.
- Interactive charts and downloadable monthly financial reports.
- Email notifications for budget alerts.
- Authentication and multi-user support.
- Integration with a vector database for long-term memory.

24. Submission Checklist

- ☐ Source code submitted.
- ☐ requirements.txt included.
- ☐ README with setup instructions.
- ☐ Architecture diagram included.
- ☐ Working Streamlit demo.
- ☐ Presentation slides prepared.
- ☐ All four AI agents implemented.
- ☐ Memory and Reflection demonstrated.

25. Client Acceptance Criteria

The project will be accepted when the application successfully demonstrates a working Supervisor-based multi-agent system capable of routing user requests, maintaining context through memory, validating responses using reflection, and providing accurate financial recommendations through a user-friendly Streamlit interface.

26. Future Enhancements

- Bank API integration for real-time transactions.
- Investment and mutual fund recommendations.
- Tax planning assistant.
- Family finance management.
- Credit score analysis.
- AI-powered retirement planning.
- Mobile application deployment.

Closing Note

FinAssist AI represents a realistic enterprise Agentic AI implementation that combines business problem solving with modern AI engineering practices. Students are encouraged to justify their design decisions, write clean modular code, and build a production-ready solution rather than focusing solely on functionality.